Agri Smart – Smart Crop Advisory System

Name : B. Likhitha
Roll No : 2520030580
Branch : CSE
Section : 4
Cluster : 1

Outline

- Abstract
- Existing System
- Proposed System
- System Architecture
- Project Modules
- Objectives
- CO Attainments-FWD
- CO Attainments-DSA
- Screenshots
- Conclusion

Abstract

Agri Smart is an intelligent agriculture management System developed to support farmers in managing their farming activities. Agriculture plays a crucial role in the economy, and farmers often face challenges such as improper crop planning, lack of weather information, and difficulty in managing farming expenses. The Agri Smart system aims to provide a digital platform where farmers can easily record and monitor their agricultural data.

Agri Smart also helps in improving productivity and reducing unnecessary expenses by providing organized data and simple analysis. The project demonstrates the use of web technologies along with data structures to create an efficient agriculture management solution.

Existing System

- Farmers mostly rely on traditional methods and manual record Keeping.
- Lack of proper digital tools for crop and farm management.
- Difficult to track expenses, crop growth, and weather information.
- Information about fertilizers, irrigation, and crop planning is often inaccurate or delayed.
- No centralized system to store and analyze farming data.
- Farmers have limited access to real-time agricultural information.
- Difficult to maintain historical records of crop production and expenses.
- Difficult to compare previous crop results to improve future farming.
- No digital system to analyze farming data for better decision making.

Proposed System

- Agri Smart provides a digital platform for agriculture management.
- Farmers can store crop information and farming records easily.
- Helps in tracking expenses related to seeds, fertilizers, and irrigation.
- Provides organized storage of agricultural data.
- Helps farmers make better farming decisions.
- Reduces manual work and improves farm productivity.
- Provides a centralized system to access agricultural information easily.
- Reduces manual paperwork and improves data accuracy.
- Supports better irrigation and fertilizer planning.
- Enables easy monitoring of crop growth and farming activities.
- Allows farmers to store and manage crop information digitally.

System Architecture

User Interface Layer :

- Provides a simple interface for farmers to enter and view agricultural data.

Input Management Layer :

- Collects crop details, soil information, and farming expenses from users.

Data Storage Layer :

- Stores agricultural data such as crops, expenses, and farming activities.

Processing Layer :

- Processes the entered data and performs required operations.

Data Analysis Layer :

- Analyzes farm records to support better farming decisions.

Algorithm Layer :

- Uses algorithms for efficient data processing and management.

Project Modules

User Registration and Login :

- Allows farmers to create and access their accounts.

Crop Management Module :

- Stores crop type, planting date, and growth details.

Weather Monitoring Module :

- Displays weather information useful for farming decisions.

Expense Management Module :

- Tracks fertilizers, seeds, irrigation, and other costs.

Data Analysis Module :

- Uses algorithms to analyze farm data and improve efficiency.

Objectives

- To develop a smart digital system for agriculture management.
- To help farmers track crop and farm activities efficiently.
- To organize agricultural data using data structures and algorithms.
- To improves farm productivity and cost management.
- To provide an easy-to-use platform for farmers.
- To maintain digital records of crops, soil conditions, and farming activities.
- To help farmers manage resources such as water, seeds, and fertilizers effectively.
- To provide accurate information for better farming decisions.
- To reduce manual record keeping and paperwork in agriculture.
- To support better crop planning and monitoring.

CO'S Attainments-FWD

CO1-HTML Structure :

- Used HTML5 semantic elements to create the webpage structure for entering trip details and displaying expense result

CO2-CSS Styling :

- Applied CSS styling, Flexbox, and responsive design to create an attractive and user-friendly interface

CO3-JavaScript Programming :

- Implemented JavaScript variables and functions to calculate food, fuel, stay expense automatically

CO4-DOM Manipulation :

- Used DOM methods and events listeners to display trip expense results dynamically.

CO5-Advanced Web Features :

- Implemented Local Storage to save trip details

CO'S Attainments-DSA

CO1-Algorithm:

- Implemented algorithms to process and manage crop details, irrigation schedules, and farming expenses efficiently.

CO2-Abstract Data Types :

- Used arrays and objects to store agricultural data such as crop information, soil details, and farm expense records.

CO3-Linear Data Structures :

- Organized crop data, farming activities, and expense records using linear data structures.

CO4-Hash-Based Structures :

- Used hashing concepts for efficient storage and retrieval of agricultural data.

CO5-Practical Applications :

- Applied data structures to manage and display farming information efficiently.

Backend Design

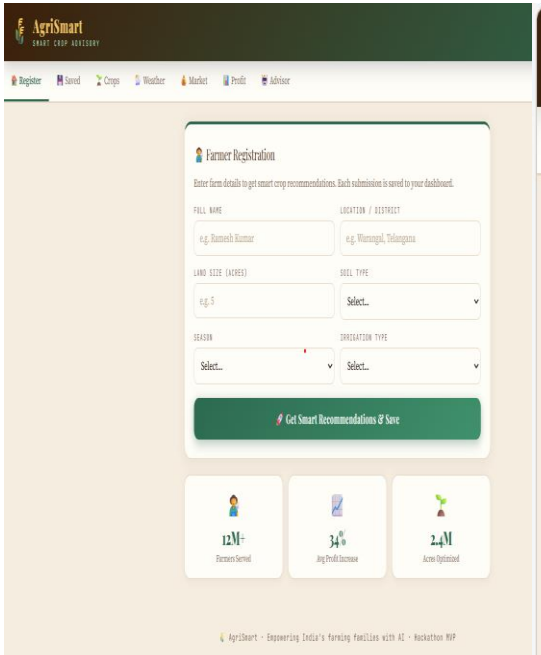
Technologies Used :

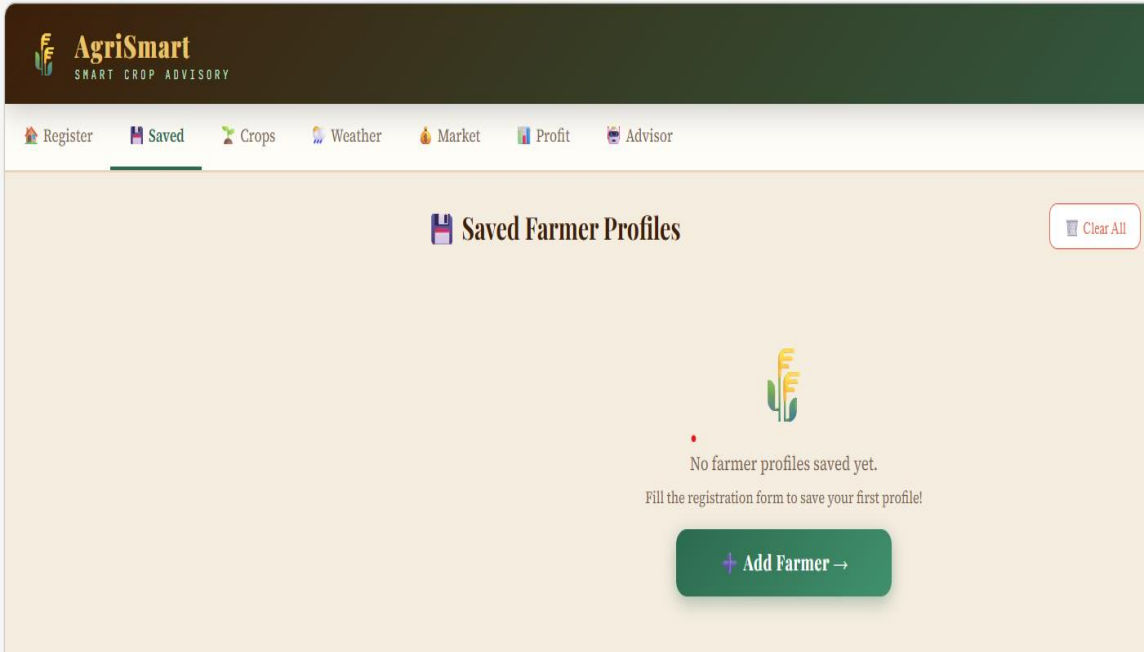
- Java
- Data Structures
- Algorithms

Key Backend Components :

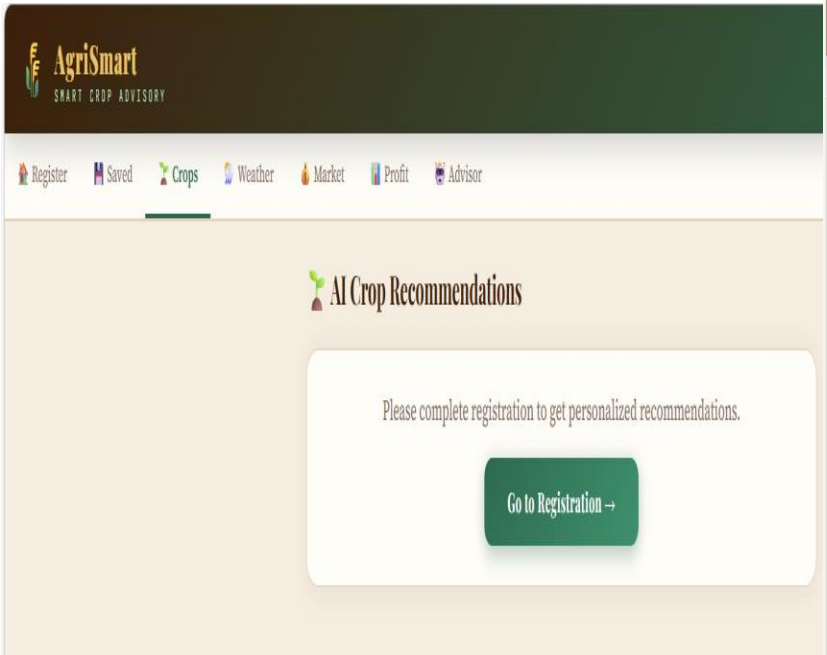
- **Crop Data Management :** Stores crop details such as crop name, planting date, and harvest information
- **Farm Expense Management :** Records expenses like seeds, fertilizers, irrigation, and labor costs.
- **Data Storage Structure :** Uses array or lists to organize agricultural data efficiently.
- **Search and Retrieval :** Uses searching algorithms to access crop and farm records quickly.
- **Result Display Module :** Displays processed farm data and analysis results.

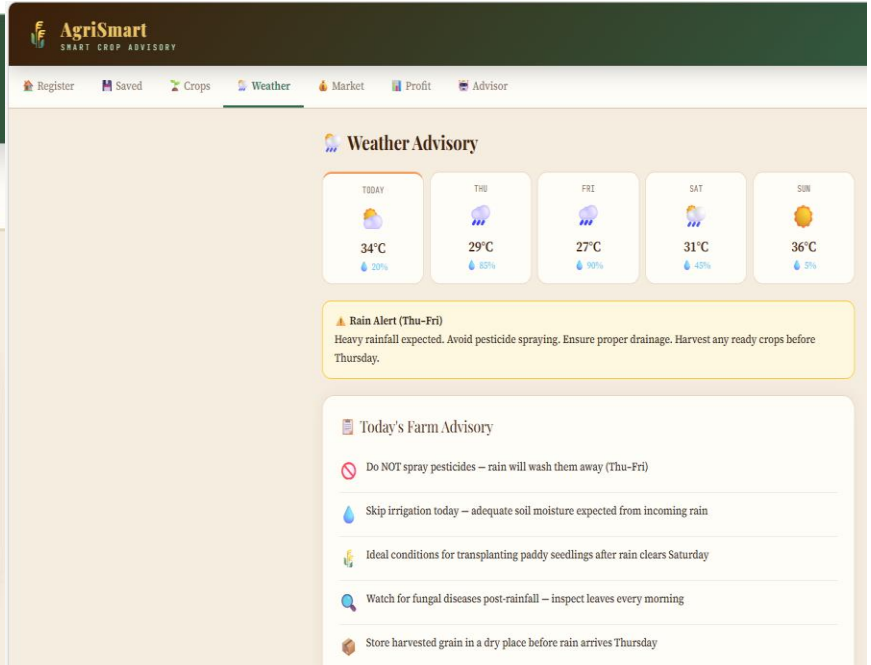
Screenshots-FWD



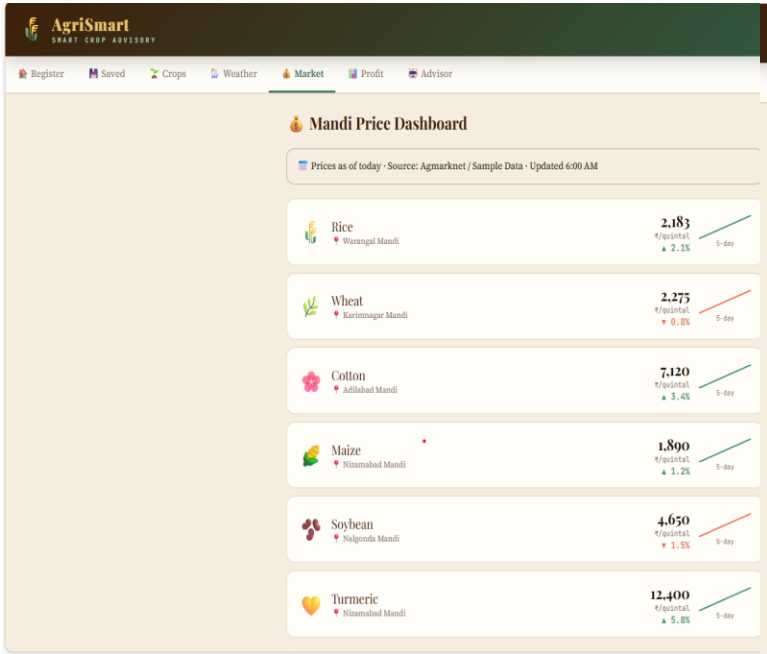


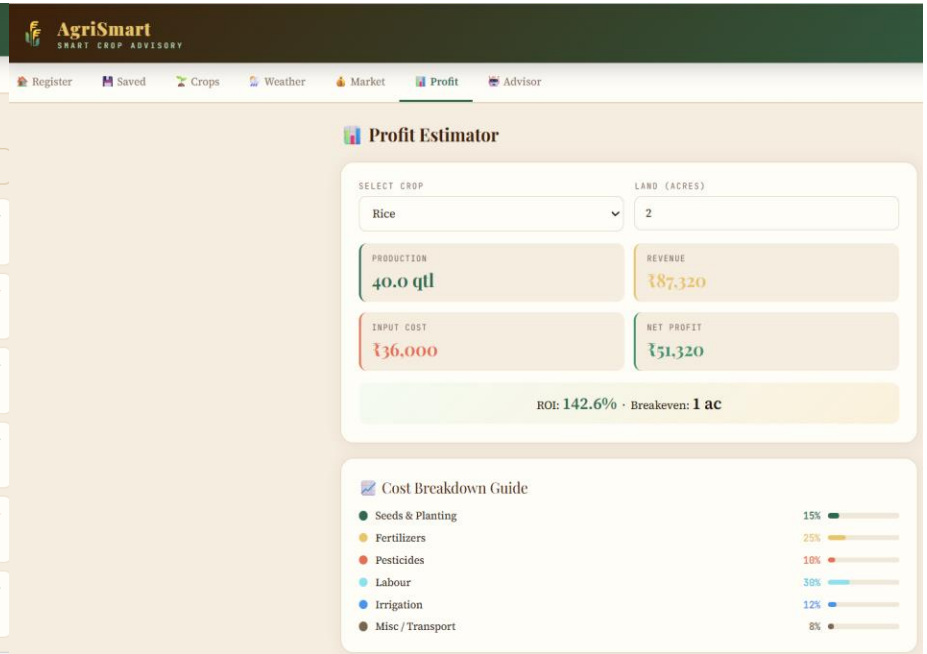
Screenshots-FWD





Screenshots-FWD





Screenshots-DSA

```java
import java.util.*;

/**
 * AGRISMART: Integrated Data Structures Project
 * Aligned with CO1 - CO6
 */
public class AgriSmartProject {

    // Global Collections for Data Management (CO4)
    private static List<FarmerProfile> savedProfiles = new
        ArrayList<>();
    private static Map<String, Double> marketRates = new
        HashMap<>();

    // ========================================
    // CO2: ABSTRACT DATA TYPES (ADTs)
    // ========================================
    static class FarmerProfile {
        String name;
        String soilType;
        double landSize;
        String timestamp;
```

```
--- AGRISMART NAVIGATION ---
1. ? HOME (Registration)
2. ? SAVED (Profiles Database)
3. ?? ADVISORY (Priority Alerts)
4. ? MARKET (Fast Lookup)
5. ? EXIT
Select Page: 1

[HOME] Farmer Registration
Enter Name: likhitha
Soil Type: black type
Land Size (Acres): 5
? Profile Saved to DS List!

--- AGRISMART NAVIGATION ---
1. ? HOME (Registration)
2. ? SAVED (Profiles Database)
3. ?? ADVISORY (Priority Alerts)
4. ? MARKET (Fast Lookup)
5. ? EXIT
Select Page: 2
```

```
artProject.java    Share    Run    Output

    public FarmerProfile(String name, String soilType,
        double landSize) {
        this.name = name;
        this.soilType = soilType;
        this.landSize = landSize;
        this.timestamp = new java.text.SimpleDateFormat("HH
            :mm:ss").format(new Date());
    }

    @Override
    public String toString() {
        return String.format("[%s] Name: %s | Soil: %s |
            Land: %.1f Acres", timestamp, name, soilType,
            landSize);
    }
}

static class Crop {
    String name;
    double profit;

    public Crop(String name, double profit) {
```

```
Select Page: 2

[SAVED] Stored Profiles (Linked List Traversal)
[16:08:37] Name: likhitha | Soil: black type | Land: 5.0 Acres

--- AGRISMART NAVIGATION ---
1. ? HOME (Registration)
2. ? SAVED (Profiles Database)
3. ?? ADVISORY (Priority Alerts)
4. ? MARKET (Fast Lookup)
5. ? EXIT
Select Page: 3

[ADVISORY] Prioritized Weather/Crop Alerts:
?? 1-CRITICAL: Cyclone Warning in coastal regions.
?? 2-HIGH: Pest attack reported in neighboring farms.
?? 3-LOW: Optimal time for fertilizer application.

--- AGRISMART NAVIGATION ---
1. ? HOME (Registration)
2. ? SAVED (Profiles Database)
3. ?? ADVISORY (Priority Alerts)
4. ? MARKET (Fast Lookup)
```

```java
// ========================================
static class AlgorithmEngine {
    // QuickSort for efficiency: O(n log n) - CO1
    public static void quickSort(List<Crop> crops, int low,
        int high) {
        if (low < high) {
            int pi = partition(crops, low, high);
            quickSort(crops, low, pi - 1);
            quickSort(crops, pi + 1, high);
        }
    }

    private static int partition(List<Crop> crops, int low,
        int high) {
        double pivot = crops.get(high).profit;
        int i = (low - 1);
        for (int j = low; j < high; j++) {
            if (crops.get(j).profit > pivot) { //
                Descending order
                i++;
                Collections.swap(crops, i, j);
            }
        }
```

```
4. ? MARKET (Fast Lookup)
5. ? EXIT
Select Page: 4

[MARKET] Live Prices (O(1) Hash Lookup)
Turmeric: ?12500.0/q
Cotton: ?7020.0/q
Rice: ?2180.0/q

Recommended Crops (Sorted by Profitability):
Turmeric - ?11000.0
Cotton - ?6500.0
Rice - ?2200.0

--- AGRISMART NAVIGATION ---
1. ? HOME (Registration)
2. ? SAVED (Profiles Database)
3. ?? ADVISORY (Priority Alerts)
4. ? MARKET (Fast Lookup)
5. ? EXIT
Select Page: 5

=== Code Execution Successful ===
```

```java
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    initializeData();

    while (true) {
        System.out.println("\n--- AGRISMART NAVIGATION ---"
            );
        System.out.println("1. 🏠 HOME (Registration)");
        System.out.println("2. 💾 SAVED (Profiles Database
            )");
        System.out.println("3. 🔮 ADVISORY (Priority
            Alerts)");
        System.out.println("4. 📈 MARKET (Fast Lookup)");
        System.out.println("5. ❌ EXIT");
        System.out.print("Select Page: ");

        int choice = sc.nextInt();
        sc.nextLine(); // Consume newline

        switch (choice) {
            case 1: showHomePage(sc); break;
            case 2: showSavedPage(); break;
            case 3: showAdvisoryPage(); break;
```

```java
// --- HOME PAGE (CO2/CO5) ---
private static void showHomePage(Scanner sc) {
    System.out.println("\n[HOME] Farmer Registration");
    System.out.print("Enter Name: "); String name = sc
        .nextLine();
    System.out.print("Soil Type: "); String soil = sc
        .nextLine();
    System.out.print("Land Size (Acres): "); double size =
        sc.nextDouble();

    FarmerProfile profile = new FarmerProfile(name, soil,
        size);
    savedProfiles.add(profile);
    System.out.println("✅ Profile Saved to DS List!");
}

// --- SAVED PAGE (CO2/CO6) ---
private static void showSavedPage() {
    System.out.println("\n[SAVED] Stored Profiles (Linked
        List Traversal)");
    if (savedProfiles.isEmpty()) {
        System.out.println("No records found.");
```

```java
// --- ADVISORY PAGE (CO3: Priority Queues) ---
private static void showAdvisoryPage() {
    // CO3: Implement Priority Queue (Heap) for urgent
        alerts
    PriorityQueue<String> alerts = new PriorityQueue<>();
    alerts.add("1-CRITICAL: Cyclone Warning in coastal
        regions.");
    alerts.add("2-HIGH: Pest attack reported in neighboring
        farms.");
    alerts.add("3-LOW: Optimal time for fertilizer
        application.");

    System.out.println("\n[ADVISORY] Prioritized Weather
        /Crop Alerts:");
        while (!alerts.isEmpty()) {
        System.out.println("⚠️ " + alerts.poll());
    }
}

// --- MARKET PAGE (CO1 & CO4) ---
private static void showMarketPage() {
    System.out.println("\n[MARKET] Live Prices (O(1) Hash
```

```
ava Compiler
```

```java
    // CO4: Fast lookup using HashMap
    marketRates.forEach((crop, price) -> System.out.println
        (crop + ": ₹" + price + "/q"));

    // CO1: Sorting crops by profit for recommendation
    List<Crop> list = new ArrayList<>();
    list.add(new Crop("Cotton", 6500));
    list.add(new Crop("Rice", 2200));
    list.add(new Crop("Turmeric", 11000));

    AlgorithmEngine.quickSort(list, 0, list.size() - 1);
    System.out.println("\nRecommended Crops (Sorted by
        Profitability):");
        for(Crop c : list) System.out.println(c.name +
            " - ₹" + c.profit);
}

private static void initializeData() {
    marketRates.put("Rice", 2180.0);
    marketRates.put("Cotton", 7020.0);
    marketRates.put("Turmeric", 12500.0);
}
```

Geo-tag pictures

Conclusion

Agri Smart is an innovative agriculture management system that helps farmers manage their farming activities in a more organized and efficient way. By providing a digital platform to store crop details, expenses, and other agricultural information, the system simplifies farm management and reduces manual work. The use of structured data management techniques allows quick access to information and helps farmers make better decisions. Overall, Agri Smart contributes to improving productivity, reducing costs, and promoting the use of technology in modern agriculture.

Agri Smart system provides a simple and effective solution for managing agricultural activities digitally. By organizing crop details, farming records, and expenses using data structures and algorithms, the system improves efficiency and reduces manual effort. This project demonstrates how technology can be used to enhance modern agriculture and make farm management more efficient.

THANK YOU